WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

Numeral Systems

Lecture 02: additive and positional systems; binary, octal and hexadecimal

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 Ways of counting

02 The radix

03 Converting between bases

04 Numbers in the DX8

05 Summary

01

Ways of counting

Numbers and numerals

Counting is an unavoidable part of human life, and people have invented many different ways of writing counts down.

> **Definition (Numeral system)**
>
> A *numeral system* is a way of writing numbers with symbols. A *numeral* is one number written in that way.

One number, four numerals:

||||||	VI	6	110_2
tally	Roman	decimal	binary

There are two big families: additive and positional systems.

Additive systems

In an additive system, each symbol has a fixed value, and the number is the sum of the symbols. Where a symbol stands does not change its magnitude.

Roman numerals

I	V	X	L	C	D	M
1	5	10	50	100	500	1000

- VI = 5 + 1 = 6
- IV = −1 + 5 = 4: a smaller symbol before a larger one is subtracted, but its magnitude is still 1
- MMXXVI = 1000+1000+10+10+5+1 = 2026

Tally marks are additive too: every mark is worth 1, wherever it is.

The trouble with additive systems

- There is no symbol for zero.
- Larger numbers need new symbols, or very long numerals:
  3888 = MMMDCCCLXXXVIII takes 15 symbols.
- Written arithmetic is awkward. Try multiplying XLVII by XXXVIII on paper.

> **Remark**
>
> Positional systems are far more useful in almost every area of mathematics and in digital logic design. From now on we work only with them.

Positional systems

In a positional system, the place of a digit decides its magnitude.

$$\begin{aligned} 123 &= 100 + 20 + 3 \\ 321 &= 300 + 20 + 1 \end{aligned}$$

The same three digits, three different numbers.

- Zero holds a place: 105 and 15 differ only by the 0.
- Our decimal digits are positional with ten digits. Other choices are just as possible.

> **Example (Base 60, still in use)**
>
> The Babylonians counted in base 60, and we still do for time: 1 h 15 min 30 s is $1 \cdot 60^2 + 15 \cdot 60 + 30 = 4530$ seconds.

02

The radix

Radix and digits

> **Definition (Radix)**
>
> The *radix*, or *base*, b of a positional system is the number of elementary symbols, the *digits*, that all its numerals are made of. The digits stand for $0, 1, \ldots, b-1$.

Each digit is multiplied by the power of b for its position, counted from 0 on the right, and the results are added:

$$\overline{d_n d_{n-1} \ldots d_0}_{\,b} = d_n \cdot b^n + d_{n-1} \cdot b^{n-1} + \cdots + d_0 \cdot b^0.$$

> **Example (Decimal, $b = 10$)**
>
> $2026 = 2 \cdot 10^3 + 0 \cdot 10^2 + 2 \cdot 10^1 + 6 \cdot 10^0$

Binary, octal and hexadecimal

System	Radix	Digits	Example
binary	2	0, 1	$101101_2 = 45$
octal	8	0, 1, …, 7	$55_8 = 45$
decimal	10	0, 1, …, 9	$45_{10} = 45$
hexadecimal	16	0, 1, …, 9, A, B, C, D, E, F	$2\mathrm{D}_{16} = 45$

- Hexadecimal needs sixteen digits, for the values 0 to 15. After 9 come the letters: A = 10, B = 11, …, F = 15.
- The value formula is the same for every radix: $\overline{b_n \ldots b_0}_{\,2} = b_n \cdot 2^n + \cdots + b_0$, and so on.

Counting in four bases

dec	bin	oct	hex
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7

dec	bin	oct	hex
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

In every base, counting works like an odometer: when a digit passes its largest value, it goes back to 0 and the next digit goes up by one ($1111_2 + 1 = 10000_2 = 16$).

Why binary?

A switch is on or off: two states, so two digits (Lecture 01).

- One binary digit is a bit. Eight bits are a byte; four bits are a *nibble*, exactly one hexadecimal digit.
- n bits have 2^n states, so they can stand for the numbers 0 to $2^n - 1$.

n	0	1	2	3	4	5	6	7	8	9	10
2^n	1	2	4	8	16	32	64	128	256	512	1024

How many bits do you need for the numbers 0 to 999?

Why binary?

A switch is on or off: two states, so two digits (Lecture 01).

- One binary digit is a bit. Eight bits are a byte; four bits are a *nibble*, exactly one hexadecimal digit.
- n bits have 2^n states, so they can stand for the numbers 0 to $2^n - 1$.

n	0	1	2	3	4	5	6	7	8	9	10
2^n	1	2	4	8	16	32	64	128	256	512	1024

How many bits do you need for the numbers 0 to 999? $2^9 = 512$ is too few and $2^{10} = 1024$ is enough: 10 bits.

Writing the base

The digits alone do not say which base is meant: is 10 ten, two, eight or sixteen?

Where	Binary	Octal	Decimal	Hexadecimal
these slides	1011_2	13_8	11	B_{16}
many programming languages	`0b1011`	`0o13`	`11`	`0xB`
Verilog (4 bits)	`4'b1011`	`4'o13`	`4'd11`	`4'hB`

The book writes $\overline{d_n \ldots d_0}_b$, with a bar over the digits. Numbers without a base are decimal.

03

Converting between bases

To decimal: expand

Multiply each digit by its power of the base and add.

$$101101_2 = 1 \cdot 32 + 0 \cdot 16 + 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 45$$

$$57_8 = 5 \cdot 8 + 7 = 47$$

$$2\mathrm{F}_{16} = 2 \cdot 16 + 15 = 47$$

> **A shortcut for binary**
>
> Every binary digit is 0 or 1, so just add up the powers of 2 where the bits are 1: $101101_2 = 32 + 8 + 4 + 1 = 45$.

To decimal: Horner's rule

Read the digits from the left. Keep one number v, starting at 0, and for each digit d: $v \leftarrow v \cdot b + d$.

Digit	v	
1	1	$= 0 \cdot 2 + 1$
0	2	$= 1 \cdot 2 + 0$
1	5	$= 2 \cdot 2 + 1$
1	11	$= 5 \cdot 2 + 1$
0	22	$= 11 \cdot 2 + 0$
1	45	$= 22 \cdot 2 + 1$

- Converting 101101_2: no powers to compute, one multiplication and one addition per digit.
- A computation in the sense of Lecture 01: the state is v, the input is the next digit.

It works because the powers can be factored out:

$$101101_2 = ((((1 \cdot 2 + 0) \cdot 2 + 1) \cdot 2 + 1) \cdot 2 + 0) \cdot 2 + 1$$

From decimal: repeated division

Divide by the base and write down the remainder; repeat with the quotient until it is 0. The remainders, read from the last to the first, are the digits.

N	$N \div 2$	remainder
45	22	1
22	11	0
11	5	1
5	2	1
2	1	0
1	0	1

- Read upwards: $45 = 101101_2$.
- Why it works: $N = q \cdot b + d_0$, so the remainder is the last digit d_0, and the quotient q has the remaining digits $d_n \ldots d_1$.
- The same method works for any base.

Example: 2026 in hexadecimal

N	$N \div 16$	remainder	
2026	126	10	A
126	7	14	E
7	0	7	7

Read upwards: $2026 = 7\text{EA}_{16}$.

Check by expanding:

$$7 \cdot 256 + 14 \cdot 16 + 10 = 1792 + 224 + 10 = 2026.$$

Common mistake

Reading the remainders from the top gives AE7_{16}, which is 2791. The first remainder is the *last* digit.

Binary to octal and hexadecimal: group the bits

Since $8 = 2^3$ and $16 = 2^4$, every octal digit is exactly 3 bits and every hexadecimal digit exactly 4 bits. Group the bits from the right.

Take 1011010111_2:

groups of 4		0010	1101	0111	
hexadecimal		2	D	7	$= 2D7_{16}$
groups of 3	001	011	010	111	
octal	1	3	2	7	$= 1327_8$

All three are 727. Pad the leftmost group with zeros if it is short.

And back again

- Hexadecimal to binary: write each digit as 4 bits.
  $C3_{16} = 1100\ 0011 = 11000011_2 = 195$
- Octal to binary: write each digit as 3 bits.
  $57_8 = 101\ 111 = 101111_2 = 47$
- Octal to hexadecimal, or back: go through binary.
  $1327_8 = 1011010111_2 = 2D7_{16}$

> **Remark**
>
> This is why hexadecimal is everywhere in computing: it is a short way of writing bits. One byte is always two hexadecimal digits, from 00_{16} to FF_{16}.

Try it

1. Write 100 in binary, octal and hexadecimal.
2. What is FF_{16} in binary and in decimal?
3. Which is larger: 1000000_2 or 77_8?

Try it

1. Write 100 in binary, octal and hexadecimal.
   `Answer:` $100 = 64 + 32 + 4 = 1100100_2$; `1 100 100` $= 144_8$; `0110 0100` $= 64_{16}$.
2. What is FF_{16} in binary and in decimal?

3. Which is larger: 1000000_2 or 77_8?

Try it

1. Write 100 in binary, octal and hexadecimal.
   `Answer:` $100 = 64 + 32 + 4 = 1100100_2$; `1 100 100` $= 144_8$; `0110 0100` $= 64_{16}$.
2. What is FF_{16} in binary and in decimal?
   `Answer:` $11111111_2 = 2^8 - 1 = 255$, the largest number in one byte.
3. Which is larger: 1000000_2 or 77_8?

Try it

1. Write 100 in binary, octal and hexadecimal.
   `Answer:` $100 = 64 + 32 + 4 = 1100100_2$; `1 100 100` $= 144_8$; `0110 0100` $= 64_{16}$.
2. What is FF_{16} in binary and in decimal?
   `Answer:` $11111111_2 = 2^8 - 1 = 255$, the largest number in one byte.
3. Which is larger: 1000000_2 or 77_8?
   `Answer:` $1000000_2 = 64$ and $77_8 = 63$, so 1000000_2, by one.

04

Numbers in the DX8

Numbers in the DX8

The computer we build in this module works with bits everywhere:

- it is an 8-bit computer: one byte holds a number from 0 to 255;
- its 8 registers are selected by 3-bit addresses, 000_2 to 111_2, because $2^3 = 8$;
- its RAM has $2^8 = 256$ bytes, so a memory address is one byte, 00_{16} to FF_{16};
- an instruction is 16 bits: four hexadecimal digits.

> **Coming back**
>
> So far every number is 0 or positive. Negative numbers in binary, and how to add them, come with two's complement `(Unit 16)`.

05

Summary

Summary

- In an additive system each symbol has a fixed value; in a positional system the place of a digit decides its magnitude.
- With radix b: digits $0, \ldots, b-1$ and $\overline{d_n \ldots d_0}_b = \sum_i d_i \cdot b^i$.
- To decimal: expand, or use Horner's rule $v \leftarrow v \cdot b + d$. From decimal: divide repeatedly, read the remainders from the last.
- Binary to octal or hexadecimal: group the bits by 3 or by 4, from the right.
- n bits give 2^n numbers, from 0 to $2^n - 1$.

Before the next lecture

- Read Section 2.1 of the book, *Numeral Systems*.
- Practise: write your year of birth in binary, octal and hexadecimal, and check each answer by converting it back.
- Count from 0 to 31 in binary. Which bit changes at every step, and which only once?

> **Next lecture** (Unit 3)
>
> Boolean algebra: an algebra with only the values 0 and 1, and its laws and theorems.